TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 5: Cây quyết định và Rừng ngẫu nhiên

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Tự tính chỉ số Gini **3**
2.1 Nhắc lại công thức . . . 3
2.2 Mười khách để tính tay . . . 3
2.3 Đọc kết quả . . . 6

3 Cây đầu tiên trên dữ liệu khách vay **6**
3.1 Đọc cây dạng chữ . . . 8
3.2 Dự đoán cho hai khách mới . . . 9

4 Cây học vẹt và cách chọn độ sâu **9**
4.1 Cây mọc tự do . . . 10
4.2 Chọn độ sâu bằng kiểm định chéo . . . 11

5 Cây không cần chuẩn hoá **12**
5.1 Đọc kết quả . . . 13

6 Rừng ngẫu nhiên **14**
6.1 Ý tưởng . . . 14
6.2 Đọc kết quả . . . 15

7 Độ quan trọng của đặc trưng **17**
7.1 Đọc kết quả . . . 18

8 Cây cho bài toán hồi quy **19**
8.1 Đọc kết quả . . . 21

9 Bài tập **21**

10 Phụ lục A, bảng tra nhanh **22**

11 Phụ lục B, lỗi thường gặp **23**

1. Mục tiêu bài thực hành

Buổi trước, KNN đoán rất tốt trên biên cong nhưng không giải thích được vì sao nó đoán như vậy. Buổi này học một mô hình ra quyết định bằng chuỗi câu hỏi có hoặc không, đọc được như luật của một nhân viên ngân hàng. Sau đó trồng cả một khu rừng gồm nhiều cây như thế để đoán chính xác hơn.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Tự tính chỉ số Gini và tự tìm câu hỏi chia dữ liệu tốt nhất bằng numpy.
2. Học một cây quyết định bằng scikit-learn, in cây ra và đọc được từng nhánh.
3. Nhận ra cây học vẹt và chọn độ sâu tối đa bằng kiểm định chéo.
4. Kiểm chứng rằng cây không cần chuẩn hoá dữ liệu.
5. Trồng rừng ngẫu nhiên và so nó với một cây đơn lẻ.
6. Đọc độ quan trọng của đặc trưng và biết một giới hạn quan trọng của nó.
7. Dùng cây cho bài toán hồi quy trên dữ liệu giá nhà.

Hai bộ dữ liệu

Bài này dùng `data/khach_vay.csv` gồm 600 hồ sơ khách vay, là số liệu mô phỏng dùng cho mục đích giảng dạy. Ngân hàng muốn đoán khách có trả nợ đúng hạn hay không. Và dùng lại `data/gia_nha.csv` của bài 1 ở phần hồi quy.

Bảng 1: *Các cột của tệp `khach_vay.csv`.*

Cột	Ý nghĩa
`thu_nhap`	Thu nhập mỗi tháng, tính bằng triệu đồng.
`ty_le_no`	Tổng nợ đang có chia cho thu nhập cả năm. Ví dụ 0.5 nghĩa là nợ bằng nửa năm thu nhập.
`co_viec_on_dinh`	Bằng 1 nếu có việc làm ổn định, bằng 0 nếu không.
`so_nguoi_phu_thuoc`	Số người khách phải nuôi, từ 0 tới 4.
`tra_dung_han`	Nhãn cần đoán. Bằng 1 nếu trả nợ đúng hạn, bằng 0 nếu trễ hạn.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai05_cay
```

Bảng 2: *Bảy tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`f1_gini.py`	Tự tính Gini và chọn câu hỏi tốt nhất
3	`f2_cay_nam_la.py`	Học và đọc một cây năm lá
4	`f3_do_sau.py`	Cây học vẹt và cách chọn độ sâu
5	`f4_khong_chuan_hoa.py`	Kiểm chứng cây không cần chuẩn hoá
6	`f5_rung.py`	Trồng rừng ngẫu nhiên
7	`f6_do_quan_trong.py`	Đọc độ quan trọng của đặc trưng
8	`f7_cay_hoi_quy.py`	Cây cho bài toán hồi quy

2. Tự tính chỉ số Gini

2.1. Nhắc lại công thức

Một câu hỏi tốt là câu hỏi tách dữ liệu thành hai nhánh, mỗi nhánh càng **thuần** càng tốt, tức là gần như chỉ chứa khách của một lớp. Để đo mức lẫn lộn của một nút, cây dùng chỉ số Gini.

Chỉ số Gini của một nút

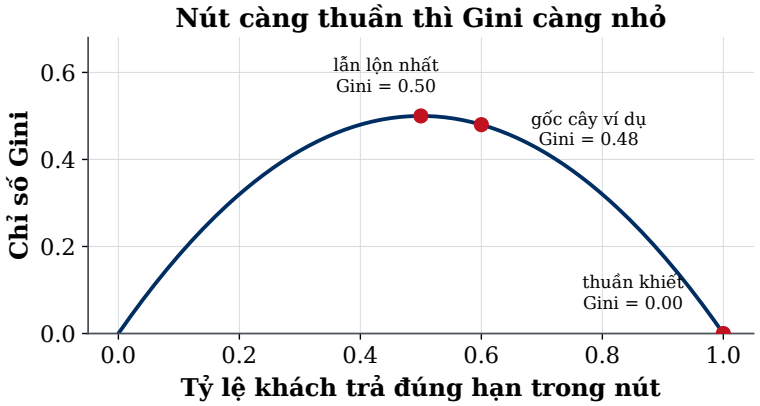
Gọi p_1 là tỷ lệ khách đúng hạn và $p_0 = 1 - p_1$ là tỷ lệ khách trễ hạn trong nút.

$$G = 1 - p_1^2 - p_0^2$$

Nút thuần khiết có $G = 0$. Nút chia đôi hai lớp có $G = 0.5$, là mức lẫn lộn cao nhất.
Khi một câu hỏi chia nút có n khách thành nhánh trái n_T khách và nhánh phải n_P khách, Gini sau khi chia là trung bình có trọng số:

$$G_{\text{chia}} = \frac{n_T}{n} G_T + \frac{n_P}{n} G_P$$

Câu hỏi tốt nhất là câu hỏi làm G_{chia} nhỏ nhất, tức là làm Gini giảm nhiều nhất.



Hình 1: *Chỉ số Gini theo tỷ lệ khách trả đúng hạn trong nút.*

2.2. Mười khách để tính tay

Trước khi đụng tới 600 hồ sơ, ta làm trên mười khách để thấy rõ từng phép tính.

Bảng 3: *Mười khách dùng để tính tay, giống bảng trên slide.*

Khách	Thu nhập	Có việc ổn định	Trả đúng hạn
1	6	0	0
2	8	0	1
3	9	0	0
4	11	1	0
5	13	1	1
6	15	1	1
7	18	1	0
8	20	1	1
9	25	1	1
10	30	1	1

Tệp mã dưới đây thử **mọi ngưỡng** của cột thu nhập, thử luôn câu hỏi về việc làm, rồi để scikit-learn tự chọn và so kết quả.

```python
# -*- coding: utf-8 -*-
"""Bước 1: tự tính chỉ số Gini và tự chọn câu hỏi tốt nhất.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""

import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

# Mười khách vay để tính tay, giống hệt bảng trên slide
nho = pd.DataFrame({
    "thu_nhap": [6, 8, 9, 11, 13, 15, 18, 20, 25, 30],
    "co_viec_on_dinh": [0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
    "tra_dung_han": [0, 1, 0, 0, 1, 1, 0, 1, 1, 1],
})
y = nho["tra_dung_han"].to_numpy()


def gini(nhan):
    """Chi so Gini cua mot nut co hai lop."""
    if len(nhan) == 0:
        return 0.0
    p1 = nhan.mean()
    return 1 - p1 ** 2 - (1 - p1) ** 2


def gini_sau_chia(di_trai):
    """Gini trung binh co trong so cua hai nhanh."""
    trai, phai = y[di_trai], y[~di_trai]
    n = len(y)
    return len(trai) / n * gini(trai) + len(phai) / n * gini(phai)

```

```

g_goc = gini(y)
print(f"Gini cua nut goc: {g_goc:.4f}")
print()

# Thử mọi ngưỡng nằm giữa hai mức thu nhập liền nhau
thu = nho["thu_nhap"].to_numpy()
muc = np.sort(np.unique(thu))
print("Cau hoi 'thu_nhap <= nguong ?'")
print("  nguong   Gini sau chia   Gini giam")
for a, b in zip(muc[:-1], muc[1:]):
    nguong = (a + b) / 2
    g = gini_sau_chia(thu <= nguong)
    print(f"  {nguong:6.1f}   {g:13.4f}   {g_goc - g:9.4f}")
print()

# Câu hỏi thứ hai: có việc làm ổn định không
viec = nho["co_viec_on_dinh"].to_numpy()
g_viec = gini_sau_chia(viec == 0)
print("Cau hoi 'co viec on dinh ?'")
print(f"  Gini sau chia {g_viec:.4f}, Gini giam {g_goc - g_viec:.4f}")
print()

# Để scikit-learn tự chọn câu hỏi đầu tiên rồi so với bảng ở trên
cot = ["thu_nhap", "co_viec_on_dinh"]
cay = DecisionTreeClassifier(max_depth=1, random_state=0)
cay.fit(nho[cot], y)
print("Cay mot tang do scikit-learn chon:")
print(export_text(cay, feature_names=cot, show_weights=True))
```

```
python code/f1_gini.py
```

```
Gini cua nut goc: 0.4800

Cau hoi 'thu_nhap <= nguong ?'
  nguong   Gini sau chia   Gini giam
     7.0         0.4000      0.0800
     8.5         0.4750      0.0050
    10.0         0.4190      0.0610
    12.0         0.3167      0.1633
    14.0         0.4000      0.0800
    16.5         0.4500      0.0300
    19.0         0.3429      0.1371
    22.5         0.4000      0.0800
    27.5         0.4444      0.0356

Cau hoi 'co viec on dinh ?'
  Gini sau chia 0.4190, Gini giam 0.0610
```

```
Cay mot tang do scikit-learn chon:
|--- thu_nhap <= 12.00
|   |--- weights: [3.00, 1.00] class: 0
|--- thu_nhap >  12.00
|   |--- weights: [1.00, 5.00] class: 1
```

2.3. Đọc kết quả

Nút gốc có 6 khách đúng hạn và 4 khách trễ hạn, nên $G = 1 - 0.6^2 - 0.4^2 = 0.48$.

Ngưỡng của cột thu nhập nằm **giữa hai giá trị liền nhau**, vì mọi ngưỡng giữa 11 và 13 đều chia mười khách y như nhau. Trong chín ngưỡng thử, ngưỡng 12.0 làm Gini giảm nhiều nhất, được 0.1633. Ngưỡng 19.0 đứng thứ hai với 0.1371.

Câu hỏi về việc làm chỉ làm Gini giảm 0.0610, kém xa. Nên câu hỏi đầu tiên của cây phải là *thu nhập có nhỏ hơn hoặc bằng 12 không*.

Phần cuối cho thấy scikit-learn chọn đúng câu hỏi đó. Dòng `weights: [3.00, 1.00]` nghĩa là nhánh trái có 3 khách trễ hạn và 1 khách đúng hạn. Số đứng trước luôn là lớp 0, số đứng sau là lớp 1.

Cây lặp lại việc này ở mọi nút

Sau khi chia nút gốc, cây làm lại đúng các bước trên cho từng nhánh con, với những khách rơi vào nhánh đó. Nó dừng khi nút đã thuần hoặc khi chạm một giới hạn ta đặt trước. Ở mỗi bước, cây chỉ chọn câu hỏi tốt nhất ngay lúc đó mà không nhìn trước các bước sau, nên người ta gọi đây là cách chọn **tham lam**.

3. Cây đầu tiên trên dữ liệu khách vay

Giờ ta để scikit-learn dựng cây trên 450 hồ sơ của tập học. Để cây còn đọc được bằng mắt, ta giới hạn nó chỉ có 5 lá bằng tham số `max_leaf_nodes=5`.

```python
# -*- coding: utf-8 -*-
"""Bước 2: học một cây nhỏ năm lá trên dữ liệu khách vay rồi đọc nó."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

df = pd.read_csv("data/khach_vay.csv")
print("Kich thuoc bang:", df.shape)
print(df.head())
print()
print("Ty le khach tra dung han:", round(df["tra_dung_han"].mean(), 2))
print()

X = df.drop(columns="tra_dung_han")
y = df["tra_dung_han"]
```

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

# Giới hạn cây chỉ được có 5 lá để còn đọc được bằng mắt
cay = DecisionTreeClassifier(max_leaf_nodes=5, random_state=0)
cay.fit(X_tr, y_tr)

print(f"Do sau {cay.get_depth()}, so la {cay.get_n_leaves()}")
print(export_text(cay, feature_names=list(X.columns),
                  show_weights=True, decimals=3))

a_hoc = accuracy_score(y_tr, cay.predict(X_tr))
a_kt = accuracy_score(y_te, cay.predict(X_te))
print(f"Accuracy tap hoc      : {a_hoc:.4f}")
print(f"Accuracy tap kiem tra : {a_kt:.4f}")
print()

# Hai khách mới, đi theo cây để biết mỗi người rơi vào lá nào
moi = pd.DataFrame({
    "thu_nhap": [20.0, 10.0],
    "ty_le_no": [0.4, 0.3],
    "co_viec_on_dinh": [1, 0],
    "so_nguoi_phu_thuoc": [2, 1],
})
du_doan = cay.predict(moi)
xac_suat = cay.predict_proba(moi)[:, 1]
for i in range(len(moi)):
    ket = "dung han" if du_doan[i] == 1 else "tre han"
    print(f"Khach {i + 1}: {ket}, xac suat dung han {xac_suat[i]:.2f}")
```

```
python code/f2_cay_nam_la.py
```

```
Kich thuoc bang: (600, 5)
   thu_nhap  ty_le_no  co_viec_on_dinh  so_nguoi_phu_thuoc  tra_dung_han
0       9.4      0.18                0                   4             0
1      15.8      0.70                0                   2             1
2       6.1      0.79                0                   0             0
3      19.4      1.13                1                   4             0
4      24.8      0.33                1                   0             1

Ty le khach tra dung han: 0.49

Do sau 3, so la 5
|--- thu_nhap <= 15.050
|   |--- ty_le_no <= 0.615
|   |   |--- co_viec_on_dinh <= 0.500
|   |   |   |--- weights: [33.000, 6.000] class: 0
|   |   |--- co_viec_on_dinh >  0.500
```

```
|   |   |   |--- weights: [35.000, 47.000] class: 1
|   |--- ty_le_no >  0.615
|   |   |--- weights: [114.000, 14.000] class: 0
|--- thu_nhap >  15.050
|   |--- ty_le_no <= 1.020
|   |   |--- weights: [27.000, 145.000] class: 1
|   |--- ty_le_no >  1.020
|   |   |--- weights: [20.000, 9.000] class: 0

Accuracy tap hoc      : 0.7978
Accuracy tap kiem tra : 0.7467

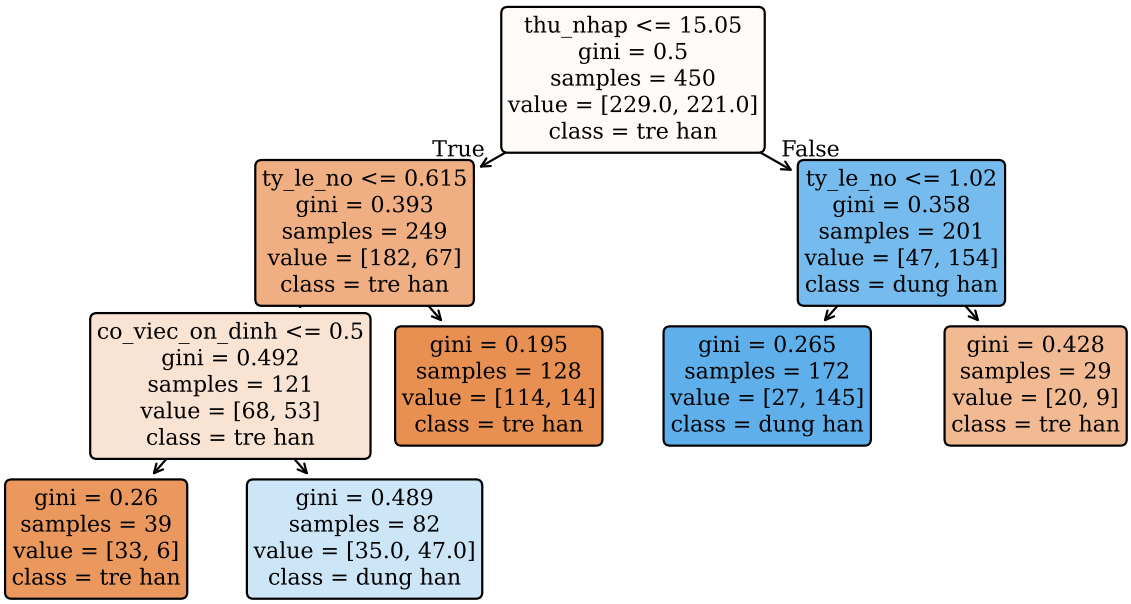
Khach 1: dung han, xac suat dung han 0.84
Khach 2: tre han, xac suat dung han 0.15
```

3.1. Đọc cây dạng chữ

Hàm `export_text` in cây thành chữ, mỗi mức thụt vào là một tầng câu hỏi. Đọc từ trên xuống:

1. Câu hỏi gốc là *thu nhập có nhỏ hơn hoặc bằng 15.05 triệu không*.
2. Nếu có, cây hỏi tiếp tỷ lệ nợ với ngưỡng 0.615. Nếu không, cây hỏi tỷ lệ nợ với ngưỡng 1.02.
3. Riêng nhóm thu nhập thấp và nợ ít, cây hỏi thêm câu thứ ba về việc làm ổn định.

Mỗi dòng `weights` ở lá cho biết số khách trễ hạn và đúng hạn của tập học rơi vào lá đó. Dòng `class` là kết luận của lá, theo lớp đông hơn.



Hình 2: *Cây năm lá vẽ bằng hàm `plot_tree`. Màu cam là lá kết luận trễ hạn, màu xanh là đúng hạn. Màu càng đậm thì nút càng thuần.*

Hình 2 vẽ lại đúng cây đó, mỗi ô ghi bốn thông tin: `gini` là chỉ số Gini của nút, `samples` là số khách, `value` là số khách của từng lớp, `class` là kết luận. Ô gốc có `gini = 0.5`, vì hai lớp

gần như chia đôi. Lá có 114 khách trễ hạn và 14 khách đúng hạn là lá thuần nhất, với `gini = 0.195`.

3.2. Dự đoán cho hai khách mới

Khách 1 có thu nhập 20 triệu và tỷ lệ nợ 0.4. Vì $20 > 15.05$ nên rẽ phải, rồi $0.4 \leq 1.02$ nên tới lá có 27 khách trễ hạn và 145 khách đúng hạn. Cây đoán **đúng hạn**.

Khách 2 có thu nhập 10 triệu, tỷ lệ nợ 0.3 và không có việc làm ổn định. Ba câu hỏi dẫn tới lá có 33 khách trễ hạn và 6 khách đúng hạn, nên cây đoán **trễ hạn**.

Xác suất của cây đến từ đâu

Hàm `predict_proba` trả về tỷ lệ khách đúng hạn trong lá mà khách mới rơi vào. Với khách 1 là $145/172 \approx 0.84$. Với khách 2 là $6/39 \approx 0.15$. Đây đúng là hai con số in ra ở cuối kết quả.

Cây nhỏ dễ đọc nhưng chưa chắc đã tốt

Cây năm lá đạt 0.7978 trên tập học và 0.7467 trên tập kiểm tra. Mục 4 sẽ tìm độ lớn phù hợp hơn cho cây.

4. Cây học vẹt và cách chọn độ sâu

Nếu không đặt giới hạn nào, cây cứ chia mãi cho tới khi mọi lá đều thuần. Tệp mã dưới đây xem chuyện gì xảy ra khi đó, rồi chọn độ sâu tối đa bằng kiểm định chéo giống cách chọn k ở bài 4.

```python
# -*- coding: utf-8 -*-
"""Bước 3: để cây mọc tự do, rồi chọn độ sâu bằng kiểm định chéo."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("data/khach_vay.csv")
X = df.drop(columns="tra_dung_han")
y = df["tra_dung_han"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

# Không đặt giới hạn nào, cây chia tới khi mọi lá đều thuần
tu_do = DecisionTreeClassifier(random_state=0).fit(X_tr, y_tr)
print("Cay moc tu do")
print(f"  do sau {tu_do.get_depth()}, so la {tu_do.get_n_leaves()}")
a_hoc = accuracy_score(y_tr, tu_do.predict(X_tr))
a_kt = accuracy_score(y_te, tu_do.predict(X_te))
print(f"  tap hoc {a_hoc:.4f}, tap kiem tra {a_kt:.4f}")
print()

```

```python
# Chọn độ sâu chỉ dựa trên tập học
print("Do sau   tap hoc   kiem dinh cheo")
diem = {}
for d in range(1, 16):
    mh = DecisionTreeClassifier(max_depth=d, random_state=0)
    diem[d] = cross_val_score(mh, X_tr, y_tr, cv=5).mean()
    hoc = accuracy_score(y_tr, mh.fit(X_tr, y_tr).predict(X_tr))
    print(f"  {d:2d}     {hoc:.4f}    {diem[d]:.4f}")
d_tot = max(diem, key=diem.get)
print()

# Chọn xong mới mở tập kiểm tra, và chỉ mở một lần
tot = DecisionTreeClassifier(max_depth=d_tot, random_state=0)
tot.fit(X_tr, y_tr)
print(f"Do sau tot nhat = {d_tot}, cay co {tot.get_n_leaves()} la")
acc = accuracy_score(y_te, tot.predict(X_te))
print(f"Accuracy tren tap kiem tra = {acc:.4f}")
```

```
python code/f3_do_sau.py
```

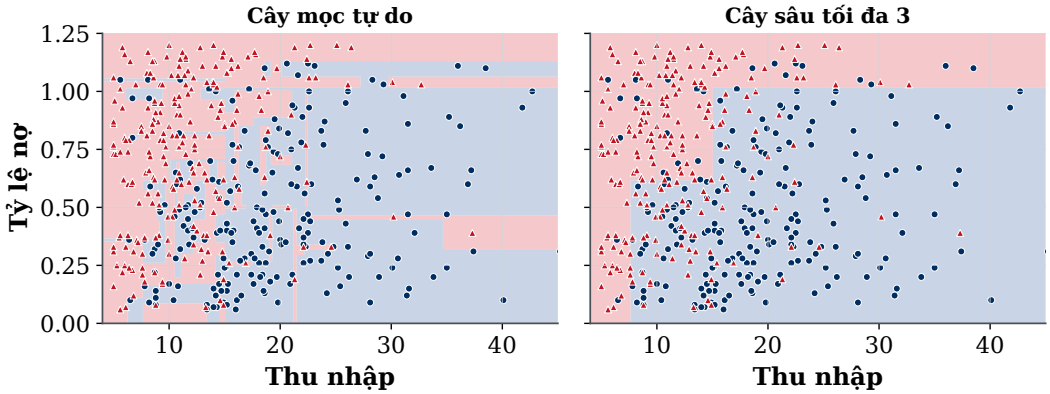
```
Cay moc tu do
  do sau 13, so la 92
  tap hoc 1.0000, tap kiem tra 0.7667

Do sau   tap hoc   kiem dinh cheo
   1     0.7467    0.7244
   2     0.7711    0.7378
   3     0.8000    0.7511
   4     0.8644    0.8089
   5     0.8978    0.8333
   6     0.9133    0.8156
   7     0.9400    0.8067
   8     0.9600    0.7733
   9     0.9800    0.7800
  10     0.9911    0.7822
  11     0.9933    0.7756
  12     0.9978    0.7778
  13     1.0000    0.7733
  14     1.0000    0.7733
  15     1.0000    0.7733

Do sau tot nhat = 5, cay co 27 la
Accuracy tren tap kiem tra = 0.8533
```

4.1. Cây mọc tự do

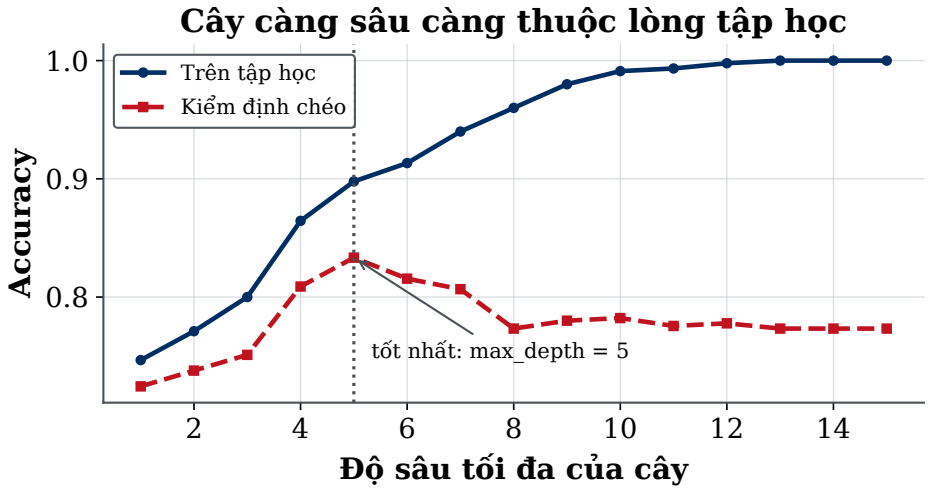
Cây tự do sâu 13 tầng và có tới 92 lá cho 450 khách, trung bình chưa tới 5 khách một lá. Nó đạt 1.0000 trên tập học nhưng chỉ 0.7667 trên tập kiểm tra.



Hình 3: *Biên quyết định của cây mọc tự do và cây sâu tối đa 3, vẽ trên hai cột thu nhập và tỷ lệ nợ.*

Hình 3 cho thấy lý do. Cây tự do bên trái cắt ra những ô chữ nhật nhỏ xíu chỉ để ôm lấy vài khách lạc loài, ví dụ những khách thu nhập cao, nợ ít mà vẫn trễ hạn. Khách mới nào rơi vào những ô đó cũng bị đoán theo đúng mấy khách lạc loài kia. Đó là hiện tượng **quá khớp**, cây đang học thuộc cả phần nhiễu của dữ liệu.

4.2. Chọn độ sâu bằng kiểm định chéo



Hình 4: *Accuracy trên tập học và điểm kiểm định chéo khi độ sâu tối đa chạy từ 1 tới 15.*

Bảng kết quả và Hình 4 kể cùng một câu chuyện:

- Cột tập học tăng đều theo độ sâu và chạm 1.0000 ở độ sâu 13.
- Cột kiểm định chéo tăng tới đỉnh 0.8333 ở độ sâu 5, rồi đi xuống và dừng quanh 0.77.

Chọn độ sâu 5. Cây lúc này chỉ còn 27 lá, và chấm một lần trên tập kiểm tra được 0.8533, cao hơn hẳn 0.7667 của cây tự do.

Độ sâu ngược chiều với k của KNN

Ở KNN, k nhỏ là học vẹt. Ở cây, độ sâu **lớn** mới là học vẹt, vì cây càng sâu thì ô chữ nhật ở lá càng nhỏ. Còn cách chọn thì giống hệt: dùng kiểm định chéo trên tập học, không bao giờ nhìn tập kiểm tra.

Bảng 4: *Các tham số dùng để cắt tỉa cây trong scikit-learn.*

Tham số	Ý nghĩa
`max_depth`	Độ sâu tối đa, số câu hỏi nhiều nhất trên một đường đi từ gốc tới lá.
`min_samples_leaf`	Mỗi lá phải có ít nhất bấy nhiêu khách.
`max_leaf_nodes`	Số lá nhiều nhất của cả cây, đã dùng ở mục 3.
`min_samples_split`	Nút phải có ít nhất bấy nhiêu khách mới được chia tiếp.

Cả bốn tham số cùng làm một việc là bắt cây dừng sớm, gọi chung là **cắt tỉa**. Bài tập ở mục 9 sẽ thử `min_samples_leaf`.

5. Cây không cần chuẩn hoá

Ở bài 4, quên chuẩn hoá làm KNN tụt xuống dưới cả mức đoán bừa. Với cây thì khác. Tệp mã dưới đây học ba cây sâu 5 trên cùng một tập học, chỉ khác cách đo các cột.

```python
# -*- coding: utf-8 -*-
"""Bước 4: kiểm chứng cây không cần chuẩn hoá dữ liệu."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("data/khach_vay.csv")
X = df.drop(columns="tra_dung_han")
y = df["tra_dung_han"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)


def cay_sau_5():
    return DecisionTreeClassifier(max_depth=5, random_state=0)


# Cây 1: dữ liệu gốc
goc = cay_sau_5().fit(X_tr, y_tr)
du_goc = goc.predict(X_te)

# Cây 2: dữ liệu đã chuẩn hoá, trung bình tính trên riêng tập học
sc = StandardScaler().fit(X_tr)
chuan = cay_sau_5().fit(sc.transform(X_tr), y_tr)
du_chuan = chuan.predict(sc.transform(X_te))

# Cây 3: đổi thu nhập từ triệu đồng sang nghìn đồng
X_tr_n, X_te_n = X_tr.copy(), X_te.copy()
```

```
X_tr_n["thu_nhap"] = X_tr_n["thu_nhap"] * 1000
X_te_n["thu_nhap"] = X_te_n["thu_nhap"] * 1000
nghin = cay_sau_5().fit(X_tr_n, y_tr)
du_nghin = nghin.predict(X_te_n)

print("Accuracy tren tap kiem tra")
print(f"  du lieu goc        : {accuracy_score(y_te, du_goc):.4f}")
print(f"  da chuan hoa       : {accuracy_score(y_te, du_chuan):.4f}")
print(f"  thu nhap nghin dong: {accuracy_score(y_te, du_nghin):.4f}")
print()
print("So khach bi doan khac cay goc")
print(f"  da chuan hoa       : {(du_chuan != du_goc).sum()}")
print(f"  thu nhap nghin dong: {(du_nghin != du_goc).sum()}")
print()
print("Nguong cua cau hoi goc")
print(f"  du lieu goc        : {goc.tree_.threshold[0]:.2f}")
print(f"  thu nhap nghin dong: {nghin.tree_.threshold[0]:.2f}")
```

```
python code/f4_khong_chuan_hoa.py
```

```
Accuracy tren tap kiem tra
  du lieu goc        : 0.8533
  da chuan hoa       : 0.8533
  thu nhap nghin dong: 0.8533

So khach bi doan khac cay goc
  da chuan hoa       : 0
  thu nhap nghin dong: 0

Nguong cua cau hoi goc
  du lieu goc        : 15.05
  thu nhap nghin dong: 15050.00
```

5.1. Đọc kết quả

Ba cây cho cùng accuracy 0.8533, và không có khách kiểm tra nào bị đoán khác đi. Chỉ có ngưỡng thay đổi: đổi thu nhập sang nghìn đồng thì ngưỡng của câu hỏi gốc đổi từ 15.05 thành 15050.

Vì sao cây không sợ thang đo

Mỗi câu hỏi của cây chỉ so **một cột** với **một ngưỡng**. Nhân cả cột với 1000 thì ngưỡng cũng nhân theo, nên khách nào ở bên trái ngưỡng vẫn ở bên trái. KNN thì khác, nó cộng bình phương chênh lệch của mọi cột lại thành khoảng cách, nên cột có thang đo lớn sẽ lấn át các cột khác.

Chuẩn hoá không có hại

Chuẩn hoá không giúp gì cho cây, nhưng cũng không làm hại. Khi so nhiều mô hình trong cùng một chương trình, cứ giữ bước chuẩn hoá trong `make_pipeline` như bài 4 cũng được.

6. Rừng ngẫu nhiên

6.1. Ý tưởng

Một cây đơn lẻ rất bấp bênh: đổi vài khách trong tập học, câu hỏi gốc có thể đổi, kéo theo cả cây bên dưới đổi theo. Rừng ngẫu nhiên trồng nhiều cây, mỗi cây học trên một phiên bản hơi khác của dữ liệu, rồi cho các cây **bỏ phiếu**. Có hai nguồn ngẫu nhiên làm các cây khác nhau:

1. **Bốc mẫu có hoàn lại**. Mỗi cây học trên 450 khách được rút có hoàn lại từ 450 khách của tập học. Có khách bị rút hai ba lần, có khách không được rút lần nào.
2. **Chọn cột ngẫu nhiên**. Ở mỗi nút, cây chỉ được chọn câu hỏi trong vài cột rút ngẫu nhiên. Mặc định là căn bậc hai của số cột, ở đây là 2 trong 4 cột.

```python
# -*- coding: utf-8 -*-
"""Bước 5: trồng rừng ngẫu nhiên và so với một cây."""

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("data/khach_vay.csv")
X = df.drop(columns="tra_dung_han")
y = df["tra_dung_han"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

# Bốc mẫu có hoàn lại: rút 450 lần từ 450 khách
rng = np.random.default_rng(0)
n = len(X_tr)
chi_so = rng.integers(0, n, n)
khac = len(np.unique(chi_so))
print(f"Boc {n} lan, co {khac} khach khac nhau ({khac / n:.0%})")
print()

rung = RandomForestClassifier(n_estimators=200, random_state=0)
rung.fit(X_tr, y_tr)
print("Rung 200 cay")
print(f"  tap hoc      : {accuracy_score(y_tr, rung.predict(X_tr)):.4f}")
print(f"  tap kiem tra : {accuracy_score(y_te, rung.predict(X_te)):.4f}")
print()

```

```python
print("So cay   accuracy tap kiem tra")
for m in [1, 10, 50, 200, 400]:
    r = RandomForestClassifier(n_estimators=m, random_state=0)
    r.fit(X_tr, y_tr)
    print(f"  {m:4d}   {accuracy_score(y_te, r.predict(X_te)):.4f}")
print()

# Bảng tổng kết ba mô hình trên cùng tập kiểm tra
ba_mo_hinh = {
    "Cay moc tu do": DecisionTreeClassifier(random_state=0),
    "Cay sau 5": DecisionTreeClassifier(max_depth=5, random_state=0),
    "Rung 200 cay": rung,
}
print("Mo hinh          tap hoc   tap kiem tra")
for ten, mh in ba_mo_hinh.items():
    mh.fit(X_tr, y_tr)
    a = accuracy_score(y_tr, mh.predict(X_tr))
    b = accuracy_score(y_te, mh.predict(X_te))
    print(f"  {ten:14s} {a:.4f}    {b:.4f}")
```

```
python code/f5_rung.py
```

```
Boc 450 lan, co 286 khach khac nhau (64%)

Rung 200 cay
  tap hoc      : 1.0000
  tap kiem tra : 0.9000

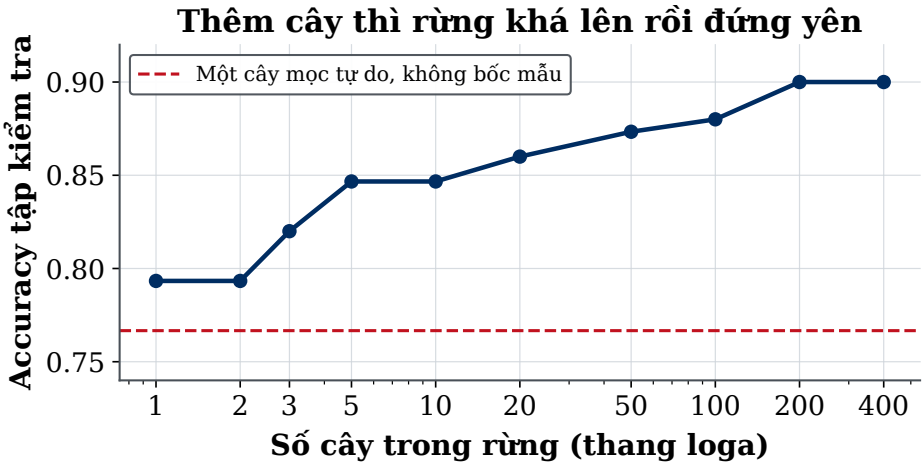
So cay   accuracy tap kiem tra
     1   0.7933
    10   0.8467
    50   0.8733
   200   0.9000
   400   0.9000

Mo hinh          tap hoc   tap kiem tra
  Cay moc tu do  1.0000    0.7667
  Cay sau 5      0.8978    0.8533
  Rung 200 cay   1.0000    0.9000
```

6.2. Đọc kết quả

Dòng đầu tiên minh hoạ một lần bốc mẫu: rút 450 lần thì chỉ được 286 khách khác nhau, khoảng 64%. Làm nhiều lần rồi lấy trung bình thì con số này rất gần 63%, nên mỗi cây trong rừng chỉ thấy khoảng hai phần ba số khách.

Rừng 200 cây cũng thuộc lòng tập học với 1.0000, nhưng trên tập kiểm tra vẫn đạt 0.9000. Bảng số cây cho thấy rừng khá lên nhanh khi thêm cây, từ 0.7933 với 1 cây lên 0.8467 với 10 cây, rồi đứng ở 0.9000 từ 200 cây trở đi.

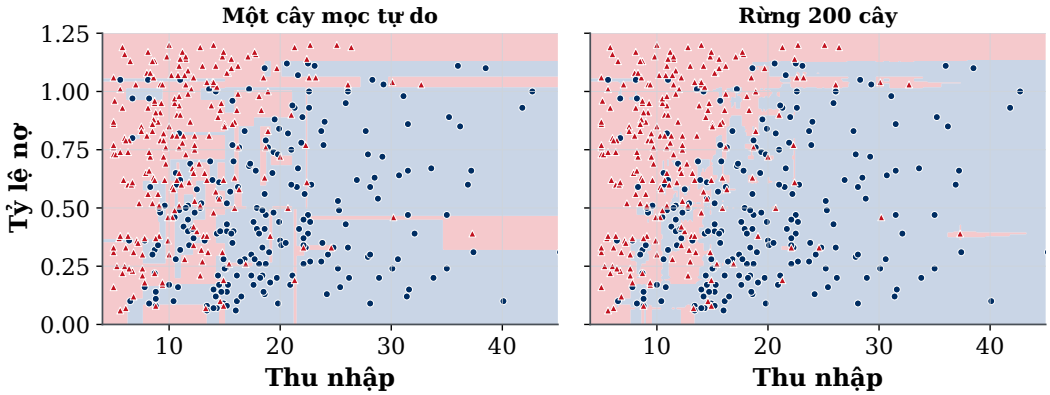


Hình 5: *Accuracy trên tập kiểm tra theo số cây trong rừng. Đường nét đứt là một cây mọc tự do.*

Bảng 5: *Ba mô hình trên cùng tập học và tập kiểm tra.*

Mô hình	Tập học	Tập kiểm tra
Cây mọc tự do	1.0000	0.7667
Cây sâu 5, chọn bằng kiểm định chéo	0.8978	0.8533
Rừng 200 cây, để mặc định	1.0000	0.9000

Trên bộ dữ liệu này, rừng thắng cả cây đã được tỉa cẩn thận mà không cần dò tham số nào.



Hình 6: *Biên quyết định của một cây mọc tự do và của rừng 200 cây, vẽ trên hai cột thu nhập và tỷ lệ nợ.*

Hình 6 giải thích vì sao. Mỗi cây trong rừng nhầm ở một chỗ khác nhau, nên khi bỏ phiếu, những ô lạc loài của từng cây bị số đông xoá đi. Ô đỏ nằm giữa vùng xanh ở bên phải của cây đơn, quanh thu nhập trên 35 triệu, gần như biến mất trong rừng.

Thêm cây không làm rừng học vẹt

Khác với độ sâu của cây, tăng số cây `n_estimators` không làm rừng quá khớp. Nó chỉ làm kết quả ổn định hơn và chạy chậm hơn. Muốn chạy nhanh hơn trên máy nhiều nhân, thêm tham số `n_jobs=-1`.

Cái giá của rừng

Không còn một cây duy nhất để đọc luật như mục 3. Muốn biết rừng dựa vào cột nào, ta phải dùng độ quan trọng của đặc trưng ở mục tiếp theo.

7. Độ quan trọng của đặc trưng

Mỗi lần một cột được dùng để chia một nút, Gini giảm đi một lượng. Cộng các lượng giảm đó của từng cột qua mọi cây trong rừng, rồi chia cho tổng, ta được **độ quan trọng** của cột. Các độ quan trọng cộng lại bằng 1.

```python
# -*- coding: utf-8 -*-
"""Bước 6: đọc độ quan trọng của đặc trưng, và thấy giới hạn của nó."""

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

df = pd.read_csv("data/khach_vay.csv")
X = df.drop(columns="tra_dung_han")
y = df["tra_dung_han"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

rung = RandomForestClassifier(n_estimators=200, random_state=0)
rung.fit(X_tr, y_tr)
qt = pd.Series(rung.feature_importances_, index=X.columns)
print("Do quan trong, bon cot that")
print(qt.sort_values(ascending=False).round(3).to_string())
print("Tong:", round(qt.sum(), 3))
print()

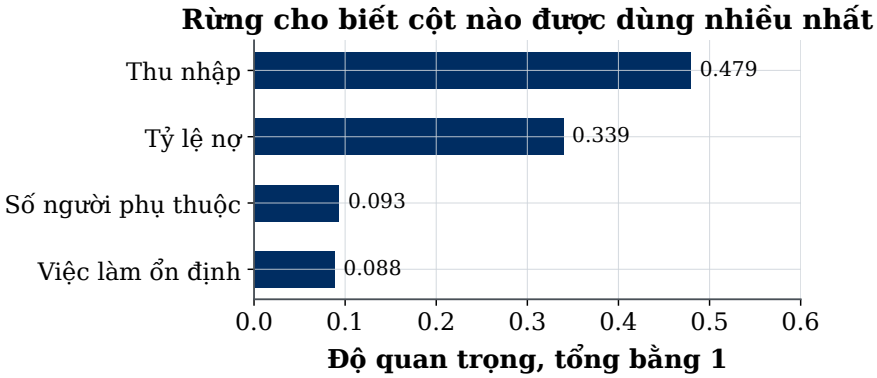
# Thêm một cột toàn số ngẫu nhiên, chẳng liên quan gì tới việc trả nợ
rng = np.random.default_rng(0)
X_tr_nn = X_tr.copy()
X_tr_nn["so_ngau_nhien"] = rng.random(len(X_tr))
rung_nn = RandomForestClassifier(n_estimators=200, random_state=0)
rung_nn.fit(X_tr_nn, y_tr)
qt_nn = pd.Series(rung_nn.feature_importances_, index=X_tr_nn.columns)
print("Do quan trong, them mot cot ngau nhien")
print(qt_nn.sort_values(ascending=False).round(3).to_string())
```

```
python code/f6_do_quan_trong.py
```

```
Do quan trong, bon cot that
thu_nhap              0.479
```

```
ty_le_no              0.339
so_nguoi_phu_thuoc    0.093
co_viec_on_dinh       0.088
Tong: 1.0

Do quan trong, them mot cot ngau nhien
thu_nhap              0.427
ty_le_no              0.270
so_ngau_nhien         0.150
co_viec_on_dinh       0.077
so_nguoi_phu_thuoc    0.076
```



Hình 7: *Độ quan trọng của bốn cột theo rừng 200 cây.*

7.1. Đọc kết quả

Với bốn cột thật, thu nhập được 0.479 và tỷ lệ nợ được 0.339, chiếm hơn bốn phần năm tổng số. Số người phụ thuộc và việc làm ổn định chỉ được khoảng 0.09 mỗi cột.

Phần thứ hai mới là chỗ đáng chú ý. Cột `so_ngau_nhien` chỉ là các số rút ngẫu nhiên, không liên quan gì tới việc trả nợ. Vậy mà nó được 0.150, xếp thứ ba, trên cả hai cột thật là việc làm ổn định và số người phụ thuộc.

Hai giới hạn của độ quan trọng

Thứ nhất, cột có **nhiều giá trị khác nhau** được lợi. Cột ngẫu nhiên có 450 giá trị khác nhau, nên cây luôn tìm được một ngưỡng nào đó tách được vài khách, và mỗi lần như vậy cột này được cộng điểm. Cột việc làm chỉ có hai giá trị 0 và 1, nên chỉ có đúng một cách chia. Thứ hai, độ quan trọng cho biết mô hình **dùng** cột nào nhiều, không chứng minh cột đó **gây ra** việc trả nợ đúng hạn.

Một cách đo đáng tin hơn

scikit-learn có hàm `permutation_importance`: xáo trộn một cột trên tập kiểm tra rồi xem accuracy giảm bao nhiêu. Cột vô dụng thì xáo trộn không làm mô hình kém đi. Bài tập 5 ở mục 9 sẽ thử cách này với cột ngẫu nhiên.

8. Cây cho bài toán hồi quy

Cây cũng đoán được một con số. Cách chia vẫn như cũ, chỉ khác ở lá: thay vì bỏ phiếu, lá đoán bằng **trung bình** giá trị của những mẫu rơi vào nó. Trong scikit-learn, lớp tương ứng là `DecisionTreeRegressor`.

```python
# -*- coding: utf-8 -*-
"""Bước 7: cây quyết định cho bài toán hồi quy, trên dữ liệu giá nhà."""

import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor, export_text

df = pd.read_csv("data/gia_nha.csv")
X = df[["dien_tich"]]
y = df["gia"]

# Cùng cách chia với bài 1 để so sánh công bằng
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          random_state=42)

cay = DecisionTreeRegressor(max_depth=3, random_state=0)
cay.fit(X_tr, y_tr)
duong = LinearRegression().fit(X_tr, y_tr)

print(f"Cay hoi quy sau 3 co {cay.get_n_leaves()} la")
print(export_text(cay, feature_names=["dien_tich"], decimals=1))

print("MAE tren tap kiem tra (ty dong)")
mae_cay = mean_absolute_error(y_te, cay.predict(X_te))
mae_duong = mean_absolute_error(y_te, duong.predict(X_te))
print(f"  Cay sau 3   : {mae_cay:.4f}")
print(f"  Duong thang : {mae_duong:.4f}")
print()

can_moi = pd.DataFrame({"dien_tich": [60.0, 100.0, 150.0, 250.0]})
print("Du doan cho bon can moi (ty dong)")
print("  dien tich     cay   duong thang")
for dt, a, b in zip(can_moi["dien_tich"], cay.predict(can_moi),
                    duong.predict(can_moi)):
    print(f"  {dt:6.0f} m2   {a:6.2f}   {b:6.2f}")
```

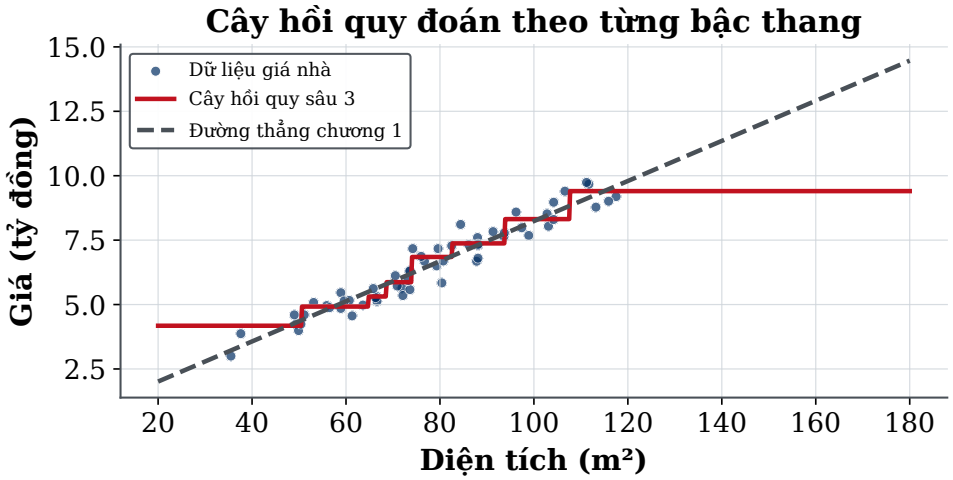
```
python code/f7_cay_hoi_quy.py
```

```
Cay hoi quy sau 3 co 8 la
|--- dien_tich <= 73.9
|   |--- dien_tich <= 64.7
```

```
|   |   |--- dien_tich <= 50.5
|   |   |   |--- value: [4.2]
|   |   |--- dien_tich >  50.5
|   |   |   |--- value: [4.9]
|   |--- dien_tich >  64.7
|   |   |--- dien_tich <= 68.5
|   |   |   |--- value: [5.3]
|   |   |--- dien_tich >  68.5
|   |   |   |--- value: [5.9]
|--- dien_tich >  73.9
|   |--- dien_tich <= 93.8
|   |   |--- dien_tich <= 82.5
|   |   |   |--- value: [6.8]
|   |   |--- dien_tich >  82.5
|   |   |   |--- value: [7.4]
|   |--- dien_tich >  93.8
|   |   |--- dien_tich <= 107.7
|   |   |   |--- value: [8.3]
|   |   |--- dien_tich >  107.7
|   |   |   |--- value: [9.4]

MAE tren tap kiem tra (ty dong)
  Cay sau 3   : 0.5039
  Duong thang : 0.2578

Du doan cho bon can moi (ty dong)
  dien tich     cay   duong thang
      60 m2     4.92     5.12
     100 m2     8.32     8.24
     150 m2     9.40    12.13
     250 m2     9.40    19.91
```



Hình 8: *Đường dự đoán của cây hồi quy sâu 3 và của đường thẳng bài 1 trên dữ liệu giá nhà.*

8.1. Đọc kết quả

Cây sâu 3 có 8 lá, mỗi lá là một mức giá, nên đường dự đoán trong Hình 8 là một **bậc thang** 8 bậc. Dòng `value: [4.2]` ở lá đầu tiên nghĩa là mọi căn nhỏ hơn hoặc bằng 50.5 m^2 đều được đoán giá 4.2 tỷ.

Trên dữ liệu giá nhà, đường thẳng thắng: MAE 0.2578 tỷ so với 0.5039 tỷ của cây. Giá nhà trong bộ dữ liệu tăng gần như thẳng theo diện tích, nên một đường thẳng mô tả đúng hơn một bậc thang.

Cây không ngoại suy được

Căn 150 m^2 và căn 250 m^2 đều được cây đoán đúng một con số 9.40 tỷ. Cả hai đều lớn hơn 107.7 m^2 nên rơi vào cùng lá cuối cùng, và lá đó chỉ biết giá trung bình của những căn rộng nhất trong tập học. Giống KNN ở bài 4, cây chỉ nói lại những gì đã thấy.

9. Bài tập

Sinh viên tạo thư mục `baitap05` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap05/bai1.py`, và trong mã vẫn viết `data/...`. Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Tự vẽ cây

Dùng hàm `plot_tree` trong `sklearn.tree` để vẽ lại cây năm lá của mục 3, rồi lưu thành tệp `cay_nam_la.png` bằng `plt.savefig`. Gợi ý, truyền thêm `feature_names`, `class_names` và `filled=True`.
Sau đó vẽ cây sâu 2 bằng `max_depth=2`. Hai lá bên trái của cây này có cùng kết luận không? Nếu có, vì sao cây vẫn chia nút đó ra làm hai?

Bài tập 2: Tự tìm câu hỏi gốc trên dữ liệu thật

Mở rộng `f1_gini.py` thành hàm `tim_cau_hoi(X, y)` thử mọi cột và mọi ngưỡng, trả về cột và ngưỡng làm Gini giảm nhiều nhất. Chạy hàm trên 450 khách của tập học.
Kết quả phải trùng với câu hỏi gốc của mọi cây ở bài này, tức là thu nhập với ngưỡng 15.05.

Bài tập 3: Gini hay entropy

Chạy lại `f3_do_sau.py` nhưng thêm `criterion="entropy"` vào `DecisionTreeClassifier`. Ghi lại độ sâu tốt nhất và accuracy trên tập kiểm tra, rồi so với khi dùng Gini ở mục 4. Nhận xét một câu: đổi thước đo có làm kết quả khác đi nhiều không?

Bài tập 4: Cắt tỉa bằng số khách tối thiểu ở lá

Giữ cây không giới hạn độ sâu, chỉ thay đổi `min_samples_leaf` lần lượt bằng 1, 2, 5, 10, 20 và 40. Chọn giá trị tốt nhất bằng kiểm định chéo 5 phần trên tập học, rồi chấm một lần trên tập kiểm tra.
So với cây sâu 5 của mục 4. Cách cắt tỉa nào cho kết quả tốt hơn trên bộ dữ liệu này?

Bài tập 5: Độ quan trọng bằng cách xáo trộn

Lấy lại rừng có cột `so_ngau_nhien` ở mục 7. Thêm một cột ngẫu nhiên tương tự vào tập kiểm tra, rồi dùng `permutation_importance` trong `sklearn.inspection` với `n_repeats=20` và `random_state=0`.
In độ quan trọng trung bình của năm cột. Cột ngẫu nhiên xếp thứ mấy theo cách đo này? So với thứ hạng của nó theo `feature_importances_`, và giải thích sự khác nhau.

Bài tập 6: Rừng cho bài toán hồi quy

Trên dữ liệu giá nhà, dùng `RandomForestRegressor` với 200 cây và `random_state=0`, cùng cách chia như mục 8. So MAE trên tập kiểm tra với cây sâu 3 và với đường thẳng.
Dự đoán giá căn 250 m^2. Rừng có ngoại suy được không? Giải thích bằng cách lá của mỗi cây đưa ra dự đoán.

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 2 và bài 5 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có hiểu cây chọn câu hỏi thế nào và đọc độ quan trọng có tỉnh táo hay không.

10. Phụ lục A, bảng tra nhanh

Bảng 6: *Các hàm mới xuất hiện trong bài 5.*

Lệnh	Thư viện	Công dụng
`DecisionTreeClassifier`	scikit-learn	Cây quyết định cho bài toán phân loại.
`DecisionTreeRegressor`	scikit-learn	Cây quyết định cho bài toán hồi quy.
`RandomForestClassifier`	scikit-learn	Rừng ngẫu nhiên cho bài toán phân loại.
`RandomForestRegressor`	scikit-learn	Rừng ngẫu nhiên cho bài toán hồi quy.
`export_text`	scikit-learn	In cây thành chữ, mỗi mức thụt vào là một tầng.
`plot_tree`	scikit-learn	Vẽ cây thành hình.
`.get_depth`, `.get_n_leaves`	scikit-learn	Độ sâu và số lá của cây đã học.
`.feature_importances_`	scikit-learn	Độ quan trọng của từng cột, tổng bằng 1.
`n_estimators`	scikit-learn	Số cây trong rừng, mặc định bằng 100.
`criterion`	scikit-learn	Thước đo độ lẫn lộn, mặc định là `"gini"`.
`np.unique`	numpy	Lấy các giá trị khác nhau, đã sắp xếp.

Bảng 7: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
Gini nút gốc của mười khách	0.48
Ngưỡng tốt nhất và mức Gini giảm	12.0 và 0.1633
Mức Gini giảm của câu hỏi về việc làm	0.0610
Cây năm lá, tập học và tập kiểm tra	0.7978 và 0.7467
Cây tự do, độ sâu và số lá	13 và 92
Cây tự do, tập kiểm tra	0.7667
Độ sâu tốt nhất và điểm kiểm định chéo	5 và 0.8333
Cây sâu 5, tập kiểm tra	0.8533
Rừng 200 cây, tập kiểm tra	0.9000
Độ quan trọng của thu nhập và tỷ lệ nợ	0.479 và 0.339
Độ quan trọng của cột ngẫu nhiên	0.150
Giá nhà, MAE của cây sâu 3 và của đường thẳng	0.5039 và 0.2578

11. Phụ lục B, lỗi thường gặp

Bảng 8: *Các lỗi hay gặp riêng của bài cây quyết định và rừng ngẫu nhiên.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
Accuracy trên tập học bằng 1.0	Cây đang mọc tự do và thuộc lòng tập học. Chấm trên tập kiểm tra, rồi giới hạn `max_depth` hoặc `min_samples_leaf`.
Chạy lại mỗi lần ra một kết quả	Quên `random_state`. Cả cây lẫn rừng đều có phần ngẫu nhiên.
`ValueError: could not convert string to float`	Có cột chữ, ví dụ *nam* và *nu*. scikit-learn chỉ nhận số, cần đổi chữ thành số trước.
Cây vẽ ra quá rối, không đọc được	Cây quá sâu. Vẽ với `max_depth=2` hoặc `max_leaf_nodes=5` để đọc, hoặc truyền `max_depth` vào chính `plot_tree`.
`export_text` in ngưỡng khác slide một chút	Mặc định chỉ in hai chữ số thập phân. Thêm `decimals=3`.
Rừng chạy rất chậm	Quá nhiều cây hoặc dữ liệu lớn. Thêm `n_jobs=-1` để dùng mọi nhân của máy.
Một cột vô dụng có độ quan trọng cao	Cột có nhiều giá trị khác nhau được lợi. Kiểm tra lại bằng `permutation_importance`.
Kết quả khác tài liệu	Quên `random_state=17` hoặc `stratify=y` khi chia dữ liệu, hoặc quên `random_state=0` khi tạo mô hình.

Một lời khuyên chung

Với dữ liệu dạng bảng, hãy thử rừng ngẫu nhiên sớm, vì nó mạnh mà ít phải dò tham số. Khi cần giải thích từng quyết định cho người khác, dùng một cây nông và in nó ra.